

## JAVA 17 OCP · YDS STUDY SERIES

# Unit 03 · Technical Memory Notes · Making Decisions

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya eksiksiz [çift dilli Chapter 3 notunu](#), bütün decision ve loop yapılarını kapsayan bir karar haritasıyla tamamlar. Ana soru şudur: **Compiler bu path'i görebiliyor mu ve runtime bu path'e gerçekten giriyor mu?** Dil çalışması: [vocabulary](#).

## Kapsam denetimi

Ana çift dilli kaynak `do/while`, enhanced `for`, switch expression, labels ve pattern variable kurallarını tam kapsar. Bu hafıza notu şu çekirdek bağları birlikte tekrar ettirir:

- switch statement ile switch expression farklarının tek tabloda görülmesi,
- selector/case type ve compile-time constant kuralları,
- loop'ların condition kontrol zamanı,
- label ile `break/continue` target kısıtları,
- pattern matching için flow scoping'in `&&||` ile ilişkisi.

### 1. `if/else`: en yakındaki eşleşme

Condition mutlaka `boolean` olmalıdır; Java numeric value'ları truthy/falsy kabul etmez.

```
int x = 1;
if (x > 0)
    if (x > 10)
        System.out.print("large");
    else
        System.out.print("small"); // nearest unmatched if
```

Braces yoksa `else`, en yakındaki unmatched `if`e bağlanır.

**Hafıza cümlesi:** `else` indentation'a değil, grammar'a bakar.

### 2. Pattern matching for `instanceof`

```
Object value = "lion";
if (value instanceof String text && text.length() > 3) {
    System.out.println(text.toUpperCase());
}
```

Pattern variable `text`, yalnız compiler'ın match'in kesin olduğunu bildiği flow path'lerinde scope içindedir. `&&` sağında kullanılabilir; aşağıdaki `||` örneğinde kullanılamaz:

```
// if (value instanceof String text || text.isEmpty()) {} // Does not compile
```

Sol taraf `false` olduğunda `||` sağ tarafı çalışabileceği için `text` henüz tanımlı olmayabilir.

### 3. Switch selector ve case kontrolü

Java 17'de klasik switch selector şu type'larla çalışır:

- `byte`, `short`, `char`, `int` ve wrapper karşılıkları,

- `String`,
- enum types.

`boolean`, `long`, `float` ve `double` klasik switch selector olamaz. `case` label'ları uygun constant expression veya enum constant olmalı; selector ile compatible ve birbirinden farklı olmalıdır.

`String`/wrapper/enum selector `null` ise standart Java 17'de runtime'da `NullPointerException` oluşur. Bu not preview özelliklerini kullanmaz.

```
final int one = 1;
int two = 2;
int selector = 1;
switch (selector) {
    case one: break; // geçerli constant variable
    // case two:      // Does not compile
}
```

## 4. Switch statement versus expression

| Özellik             | Switch statement               | Switch expression             |
|---------------------|--------------------------------|-------------------------------|
| Yazım               | Colon veya arrow               | Colon veya arrow              |
| Fall-through        | Colon biçiminde mümkün         | Colon biçiminde mümkün        |
| Değer üretme        | Hayır                          | Normal tamamlanırsa zorunlu   |
| Bloktan değer verme | <code>yield</code> kullanılmaz | <code>yield</code> kullanılır |
| Exhaustive olma     | Zorunlu değil                  | Zorunlu                       |

Arrow (`->`) her iki yapıda da fall-through yapmaz. **Yazım biçimiyle statement/expression ayrımını birbirine eşitleme.** [JLS 17: switch statement](#), [JLS 17: switch expression](#).

```
String season = "SPRING";
int length = switch (season) {
    case "WINTER", "SUMMER" -> 6;
    case "SPRING" -> {
        int base = 3;
        yield base + 3;
    }
    default -> 4;
};
```

Switch expression'dan normal çıkılan her path bir value üretmelidir; bir path exception fırlatarak da sonlanabilir. Enum'un tüm constants'ı kapsanıyorsa explicit `default` her zaman zorunlu değildir.

**Hafıza cümlesi:** Colon akabilir; arrow tek kola girer. Expression sonuçsuz çıkamaz.

## 5. Loop zaman çizelgesi

| Loop                      | İlk condition'dan önce body çalışabilir mi?                      | Tipik kullanım                 |
|---------------------------|------------------------------------------------------------------|--------------------------------|
| <code>while</code>        | Hayır                                                            | iteration sayısı bilinmiyor    |
| <code>do/while</code>     | Evet, önce gövdeye girilir                                       | body en az bir kez gerekli     |
| basic <code>for</code>    | Hayır                                                            | init/condition/update birlikte |
| enhanced <code>for</code> | Boolean condition yazılmaz;<br>array/Iterable elemanları gezilir | index gerekmiyor               |

Basic `for` akışı:

```
initializer → condition → body → update → condition → ...
```

Initializer ve update bölümleri birden fazla expression taşıyabilir; declaration varsa aynı statement'taki variable'lar compatible ortak type kullanmalıdır.

```
for (int i = 0, j = 3; i < j; i++, j--) {}
```

Enhanced `for` yalnız array veya `Iterable` üzerinde çalışır:

```
for (var item : items) {
    System.out.println(item);
}
```

Loop variable'a assignment yapmak array/collection elemanını otomatik değiştirmez.

## 6. Labels, `break`, `continue`, `return`

- Unlabeled `break`, nearest loop veya switch'ten çıkar.
- Labeled `break`, adı verilen labeled statement'ı sonlandırabilir.
- `continue`, yalnız loop target'ına gider; labeled target da loop olmalıdır.
- `return`, current method'u tamamen bitirir.

```
OUTER: for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) continue OUTER;
    }
}
```

**Hafıza cümlesi:** `break` dışarı çıkar; `continue` sonraki turu çağırır; `return` method'u kapatır.

## 7. Scope ve unreachable code

Bir block'ta bildirilen variable o block dışından erişilemez. `for` initializer variable'ı yalnız loop header/body kapsamındadır. Compiler kesin biçimde hiçbir zaman ulaşılamayan bazı statement'ları reddeder:

```
while (true) {  
    break;  
    // System.out.println("never"); // unreachable  
}
```

Infinite loop her zaman compilation error değildir; exit path gerekip gerekmediği çevredeki statement'lara ve reachability kurallarına bağlıdır.

## OCP için 20 saniyelik analiz sırası

- 1 Önce compile-time scope: variable bu path'te erişilebilir mi?
- 2 Condition gerçekten boolean mı?
- 3 Switch case constant/compatible/unique mi?
- 4 Switch expression exhaustive ve bütün path'ler value üretiyor mu?
- 5 Loop'ta condition ne zaman, update ne zaman çalışıyor?
- 6 `break/continue` hangi exact target'a gidiyor?

## Aktif hatırlama · Özgün çalışma soruları

- 1 Pattern variable neden `instanceof ... || ...` sağında genellikle kullanılamaz?
- 2 Switch expression ile `yield` ne zaman gerekir?
- 3 Hangi loop body'yi condition'dan önce çalıştırır?
- 4 Labeled `continue` herhangi bir labeled block'a gidebilir mi?

## Cevaplar

- 1 Sol match `false` iken sağ taraf çalışabilir; pattern variable oluşmamıştır.
- 2 Switch expression içinde bir bloktan değer verirken; colon biçimi de `yield` kullanabilir.
- 3 `do/while`.
- 4 Hayır; target bir loop olmalıdır.